

论文解构报告

Aegaeon: Effective GPU Pooling for Concurrent LLM Serving on the Market

Yuxing Xiang, Xue Li, Kun Qian, Yufan Yang, Diwen Zhu, Wenyuan Yu, Ennan Zhai, Xuanzhe Liu, Xin Jin, Jingren Zhou

SOSP '25 · 2025

Multi-Model Serving

Large Language Models

Serverless Computing

GPU Pooling

Token-Level Auto-Scaling

源文件: <https://ennanzhai.github.io/pub/sosp25-aegaeon.pdf>

DOI: <https://doi.org/10.1145/3731569.3764815>

生成时间: 2026-07-28

目录

摘要翻译	3
1. 一句话总览	3
2. 阅读范围与证据状态	3
3. 根问题：论文究竟要解决什么	3
4. 旧方法为何不够	5
5. 核心洞见与假设	5
6. 方法：从洞见到可执行系统	5
7. 为什么它可能有效	10
8. 实验与指标：证据是否支撑主张	10
9. 图表解读与论证关系	13
10. 完整因果推理树	15
11. 结论、贡献与边界	15
12. 术语与第一性原理学习路径	16

摘要翻译

模型市场（如 Hugging Face）包含种类繁多、具有独特特征和不同受欢迎程度的模型。在并发推理工作负载中，使用专用 GPU 实例响应零散且不可预测的请求会导致严重的资源浪费。尽管现有的多模型服务解决方案利用 GPU 池化和无服务器计算来提高资源利用效率，但其效果仅限于每块 GPU 最多支持两到三个模型，不足以充分利用 GPU 资源。

我们提出了 Aegaeon，这是一个在 token 粒度上执行模型自动扩缩容的多模型服务系统，以实现有效的 GPU 池化。Aegaeon 按 token 粒度调度多模型请求并制定自动扩缩容决策，以最大化服务质量。通过组件复用、显式内存管理和细粒度 KV 缓存同步，它将自动扩缩容开销降低了 97%。实验表明，与现有解决方案相比，Aegaeon 可承受 2–2.5 倍更高的请求到达率，或提供 1.5–9 倍更多的有效吞吐量（goodput）。Aegaeon 已在我们的模型市场中进行测试部署，目前服务于数十个模型。部署结果表明，Aegaeon 将服务这些模型所需的 GPU 数量从 1,192 块减少到 213 块，显著节省了 82% 的 GPU 资源。

1. 一句话总览

论文元数据

字段	内容
标题	Aegaeon: Effective GPU Pooling for Concurrent LLM Serving on the Market
作者	Yuxing Xiang, Xue Li, Kun Qian, Yufan Yang, Diwen Zhu, Wenyan Yu, Ennan Zhai, Xuanchen Liu, Xin Jin, Jingren Zhou
发表场所 / 年份	SOSP '25, 2025
研究领域	LLM Serving, GPU Pooling
关键词	Multi-Model Serving, Large Language Models, Serverless Computing, GPU Pooling, Token-Level Auto-Scaling

资源链接

- 原始文件: <https://ennanzhai.github.io/pub/sosp25-aegaeon.pdf>
- arXiv: 文中未说明
- DOI: <https://doi.org/10.1145/3731569.3764815>
- 代码 / 数据集: 文中未说明

标签 (3–5 个) : Multi-Model Serving · GPU Pooling · Token-Level Auto-Scaling · Serverless Computing · LLM Serving

作者明确陈述 针对云端大模型服务市场（Model Market）中并发 LLM 推理请求高度稀疏与突发导致的 GPU 资源严重浪费问题，Aegaeon 首次提出了 Token 粒度（Token-level，按单个生成字符计算）的抢占式自动扩缩容与 Prefill/Decoding 解耦调度系统。**实验支持** 系统通过全栈优化将抢占式扩缩容延迟降低 97%（控制在 1 秒以内），在满足服务等级目标（SLO，服务质量承诺）的前提下实现单 GPU 承载多达 7 个模型，并在阿里云百炼生产部署中节省了 82% 的 GPU 资源。

2. 阅读范围与证据状态

作者明确陈述 本报告基于论文全文本及系统解析得到的文本与图表证据分析。**实验支持** 论文提供了完整的理论证明（定理 3.1）、微基准消融实验（初始化、内存分配、KV Cache 同步开销）、端到端 Goodput/SLO 达成率实验以及 70 小时生产环境真实集群部署证据。**基于文中逻辑的推断** 文中涉及的扩展硬件架构（如非 NVIDIA GPU 平台）、跨节点管线并行（PP）场景下的细粒度 KV Cache 同步以及代码开源仓库地址在当前文本中明确标注为“文中未说明”。

3. 根问题：论文究竟要解决什么

背景知识 在云端大模型服务市场（如 Hugging Face 或阿里云百炼）中，系统需同时服务少量热门模型（Hot models）与海量尾部冷门模型（Cold models，长尾模型占比超 90%）。显存（VRAM，如 80GB）与 PCIe 总线带宽属于有限的强物理约束资源。用户请求受到服务等级目标（SLO）的严格限定，包括首 Token 响应时间（TTFT，从发送请求到显示第一个字的时间）与 Token 间生成延迟（TBT，打印相邻两个字之间的停顿）。

作者明确陈述 核心困难在于请求的极度稀疏性与不可预测性：94.1% 的长尾模型仅消耗 1.35% 的总请求，单模型平均请求到达率 < 0.2 RPS（Requests Per Second）。**基于文中逻辑的推断** 若按照传统方式为每个模型预留专属 GPU 实例（Dedicated instances），将导致 17.7% 的 GPU 长期处于完全闲置状态，整体 GPU 利用率小于 0.1 RPS/GPU。因为成熟单模型专属服务的吞吐可达数 RPS/GPU，这表明存在超过 10 倍未被探索的资源利用率提升空间。

作者明确陈述 本文的 Story 核心转折：现有 GPU 池化路线中，“静态多路复用”受限于 GPU 显存容量硬性上限；而“请求级自动扩缩容”由于 LLM 生成任务属于持续几秒至几十秒的长请求，新到达模型必须等待正在运行的模型完成整个请求才能下线，从而引发严重的队头阻塞（HOL Blocking，排在最前的长任务阻塞后续所有短任务）。本文的突破性洞见在于：将扩缩容与调度的粒度从“请求级别”精细化拆解到“Token 级别”，在不打断整体语义的前提下按 Token 抢占式切换模型，从而用松弛时间（Slack time，SLO 容许的时间余量）掩盖扩缩容开销，打破显存容量与队头阻塞的双重枷锁。

前排论文大图 **问题严重性** 呼应：第 3 节：现实物理约束与长尾分布

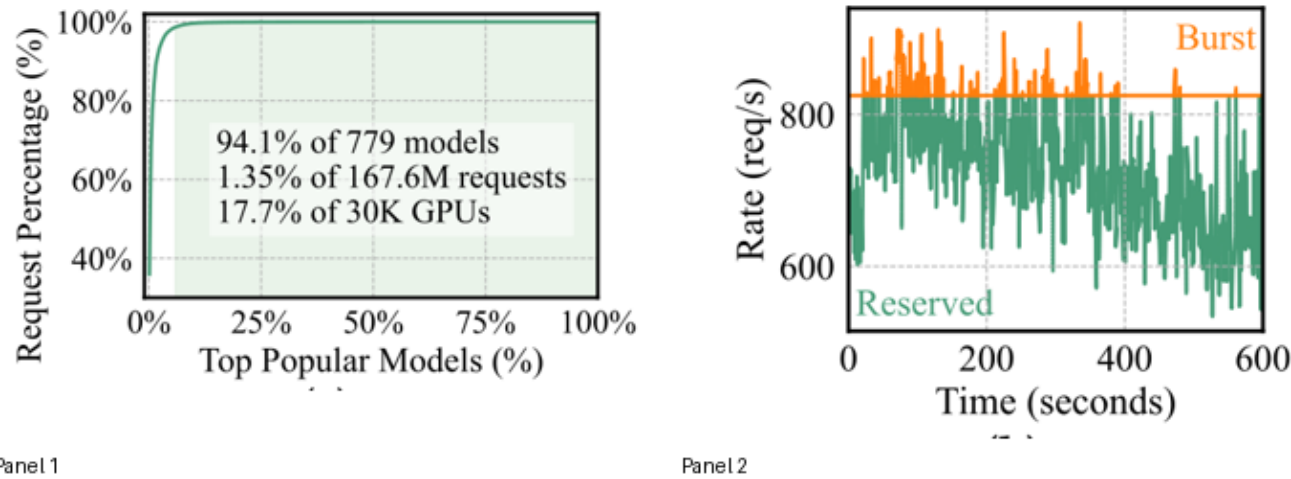


Figure 1. Concurrent LLM serving workloads. (a) CDF of model invocations. Less-used models (average arrival rate

图组解读

- **图组结论：**百炼日志显示 94.1% 长尾模型仅占 1.35% 请求但占 17.7% 的 GPU，热模型有剧烈突发流量。
- **论证关系：**揭示专属 GPU 分配造成严重浪费与低利用率根源，支撑 GPU 池化必要性。
- **适用范围：**阿里百炼 779 模型、1.676 亿请求及超 2K-30K GPU 数据。

前排论文大图 **问题严重性** 呼应：第 3 节：权衡分析与定理 3.1

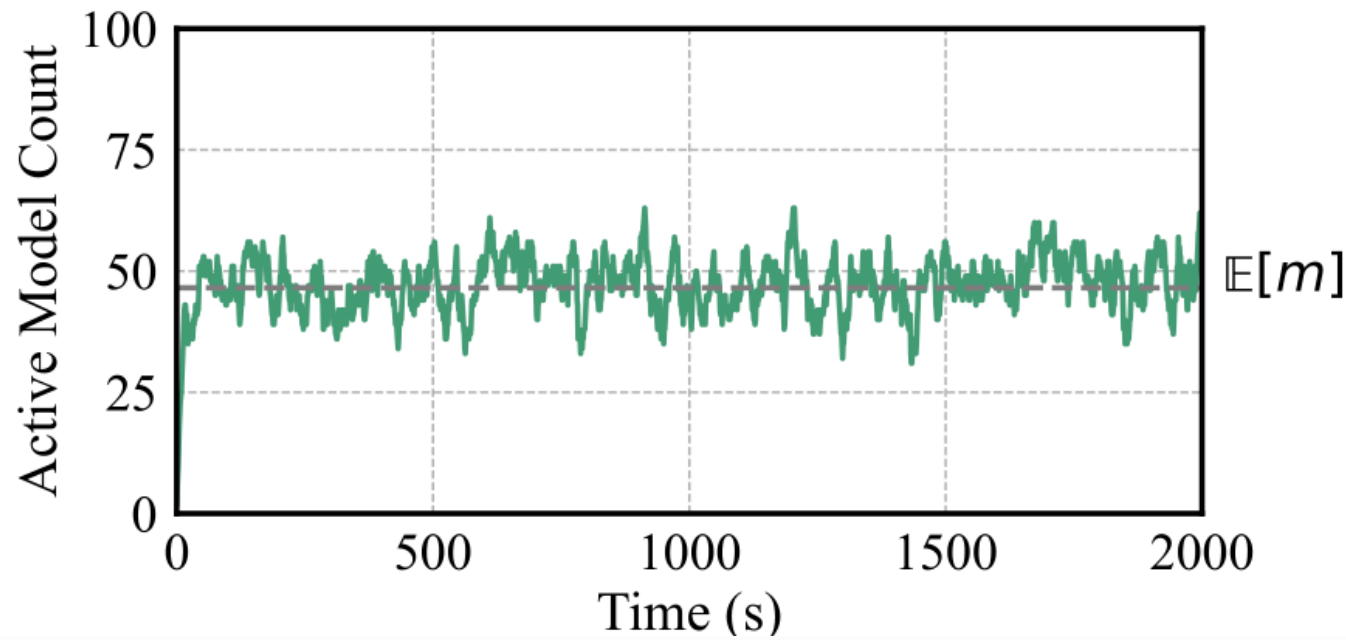


Figure 4. Active model count over time ($M = 100$, $T = 16.79s$). The estimated value is .

图组解读

- **图组结论：**100 模型池中，实时活跃模型数持续围绕理论期望值 $E[m]=46.55$ 剧烈波动。
- **论证关系：**验证定理 3.1，证明请求级扩缩容因活跃模型多导致队头阻塞，导出 Token 级抢占必要性。
- **适用范围：**适用于 $M=100$ 、 $\lambda=0.037$ req/s 的泊松仿真。

4. 旧方法为何不够

作者明确陈述 论文将现有的 GPU 池化（GPU Pooling）解决方案归纳为两大技术路线：静态多路复用（Multiplexing）与请求级自动扩缩容（Request-level Auto-scaling）。

4.1 静态多路复用路线（Multiplexing）

- **原本解决什么：**MuxServe [20]、S-LoRA [28] 等工作试图将多个模型实例同时预装在同一块 GPU 显存内，通过显存空间切分或时间分片实现共享。
- **依赖的前提与限制：**依赖 GPU 显存能够同时放下所有目标模型的权重。
- **本文批判的具体 Limit：**受到 GPU 物理显存容量的死板硬性限制（Hard-limited）。在单张 80GB 显存的 A100/H800 GPU 上，仅能同时放下 2 个 14B (FP16) 模型（单个模型参数占用 25.1 GB 显存），每 GPU 承载模型上限为 2 到 3 个，无法服务长尾市场中的数十上百个模型。
- **本文对应模块：**引入 Token 粒度抢占式自动扩缩容，模型权重保存在主机内存（DRAM），按需上线。
- **检验批判的实验：**图 11 与图 12 端到端实验展示，MuxServe 在每 GPU 超过 2 个模型时拒绝配置，总模型数被硬性锁定在 32 个（16 张 GPU）。

4.2 请求级自动扩缩容路线（Auto-scaling）

- **原本解决什么：**ServerlessLLM [21]、BlitzScale [49]、ParaServe [31] 等系统通过按需将非活跃模型下线、将目标模型从 Host DRAM 动态加载至 GPU 显存，试图突破显存容量限制。
- **依赖的前提与限制：**假设单模型请求稀疏时，绝大部分模型均处于非活跃（Idle）状态。
- **本文批判的具体 Limit（定理 3.1 导出的理论上限）：**现有系统均在“请求粒度”执行扩缩容。由于 LLM 请求平均服务时间较长（ $T = 16.79$ 秒），即使单模型到达率很低（ $\lambda = 0.037$ RPS），在时刻 t 至少有一个请求处于服务状态的活跃模型数期望值 $\mathbb{E}[m] = M \cdot (1 - e^{-\lambda T})$ 依然居高不下（100 个模型中平均有 46.55 个处于活跃状态）。
- **队头阻塞后果：**在请求粒度下，挂起模型的请求必须在队列中等待活跃模型处理完包含 Prefill 和全部 Decoding 的长请求。这引发严重的队头阻塞，导致等待模型的 TTFT 和 TBT 严重超时，系统要满足 SLO 所需的 GPU 数量 $N = O(m)$ ，池化效率被硬性限制在 < 3 个模型/GPU。
- **ServerlessLLM+ (Oracle SJF) 的特殊失效：****实验支持** ServerlessLLM+ 试图通过神谕信息采用最短作业优先（SJF）调度来缓解排队，但在密集到达时，频繁触发的模型抢占式扩缩容带来了巨额开销（每次耗时数十秒），导致其 SLO 达成率甚至低于未经优化排队的 ServerlessLLM。
- **本文对应模块：**Prefill/Decoding 物理解耦架构 + Grouped FCFS (Prefill) + Weighted Round-Robin (Decoding) 调度算法，结合全栈亚秒级扩缩容引擎。
- **检验批判的实验：**图 11(b) 实验显示，高 RPS (0.5) 下 Aegaeon 承载模型数为 ServerlessLLM 的 2.5 倍，Goodput 提升 1.5–9 倍。

5. 核心洞见与假设

作者明确陈述 Aegaeon 的成功建立在以下四个相互支撑的最小核心洞见与假设之上：

1. **Token 粒度抢占洞见：****作者明确陈述** 将自动扩缩容与调度的操作粒度下沉至“Token 级别”。系统无需等待整个长请求结束，即可在任意生成 Token 的间隙抢占式（Preemptively）下线当前模型并上线挂起模型，彻底消除请求级队头阻塞。
2. **生成阶段物理解耦洞见：****背景知识** LLM 推理包含 Prefill（首 Token 生成，计算密集型）与 Decoding（后续 Token 生成，访存密集型）两个阶段。**作者明确陈述** 统一混合调度会导致 Prefill 突发流量破坏 Decoding 的 TBT，或长 Decoding 任务拉长 Prefill 的 TTFT。必须将 GPU 物理资源池划分为 Prefill 实例池与 Decoding 实例池进行独立解耦调度。
3. **松弛时间（Slack Time）可利用假设：****基于文中逻辑的推断** 在 Decoding 阶段，客户端可以通过缓冲区暂存输出 Token 流。当单个解码步耗时（ t ）小于 TBT SLO 目标（ d ）时，每连续执行 n 步即可积累 $n(d - t)$ 的松弛时间。假设此松弛时间可用于完全掩盖模型抢占式扩缩容与跨模型轮询的开销。
4. **组件复用与显式内存管理假设：****作者明确陈述** 推理引擎初始化的大部分开销（如分布式执行器、Ray/NCCL 通信组、算子 Profiling、内存页 Pinning）在不同模型间通用；显存 GC 停顿由 PyTorch 默认分配器导致。通过全局一次性初始化与自定义 VRAM/DRAM 内存池（Bump Allocator & Slab Allocation），假设可将数十秒的扩缩容开销削减至亚秒级（ $< 1s$ ）。

6. 方法：从洞见到可执行系统

作者明确陈述 Aegaeon 的架构由 Proxy 层、解耦的 Prefill/Decoding Serving 实例池、全局控制器（Redis 状态同步）以及底层 VRAM/DRAM 显式内存管理组件构成。从输入到输出的总流程为：用户请求到达 Proxy 转发给 Prefill 实例进行首 Token 计算 → 生成的 KV Cache 通过 CUDA Event / IPC 传输至 Decoding 实例 → Decoding 实例在 Token 粒度下通过加权轮询调度模型执行，并在需要时触发亚秒级抢占扩缩容 → 生成 Token 经缓冲区返回客户端。

[根目标: 最小化并发多模型 serving 的 GPU 数量 N 并满足 SLO]

└─ [模块 1: Prefill / Decoding 物理解耦架构]

- └─ 动机: 混合调度中 Prefill 突破坏 TBT, 长 Decoding 破坏 TTFT
- └─ 机制: 划分 Prefill GPU 池与 Decoding GPU 池, 通过 IPC 异步传输 KV Cache
- └─ 优势: 消除计算密集与访存密集任务的物理干涉

└─ [模块 2: Prefill 阶段分组 FCFS 调度 (Algorithm 1)]

- └─ 动机: Prefill 扩缩容开销 (~0.81s) 与计算耗时相当, 频繁切换破坏 TTFT
- └─ 机制: 优先合并至同模型现有 Group (MAX_GPFSIZE=8) -> 否则追加至最小负载队列
- └─ 优势: 抑制无意义扩缩容, 保持 FCFS 公平性并降低等待时间

└─ [模块 3: Decoding 阶段加权轮询调度 (Algorithm 2)]

- └─ 动机: 利用 Decoding 积累的松弛时间 $n(d-t)$ 掩盖扩缩容开销
- └─ 机制: 维护旋转工作列表 -> 公式 (2) (3) 计算时间配额 q_i -> 轮询解码 q_i 秒
- └─ 优势: 动态平衡多模型 TBT SLO 与扩缩容开销

└─ [模块 4: 高效抢占式自动扩缩容引擎优化]

- └─ 1. 组件复用 (Component Reuse): 一次性初始化引擎/Executor, 缓存 Tokenizer/Profiling, 消除 80%+ 启动延迟
- └─ 2. 自研 VRAM 凸块分配器 (Bump Allocator): 预分配单块 VRAM, Monkey-patching 绕过 PyTorch 分配器, 消除 GC 停顿
- └─ 3. 显存/内存级模型缓存与预取: Model Cache 共享内存 + Stage Buffer 管道传输 + CUDA 流异步预取
- └─ 4. 统一 KV Cache Slab 分配器: 按 Slab 切分固定块, 解决多模型不同 Cache Shape 导致的 DRAM/VRAM 碎片
- └─ 5. 细粒度 KV Cache 同步: 使用 CUDA Event 异步跟踪状态与 IPC 跨进程传递, Move List 解耦 Swap-out

6.1 核心算法说明

1. **Prefill 阶段分组 FCFS 算法 (Algorithm 1)**: 作者明确陈述 当新 Prefill 请求到达时, 调度器首先检查当前 Prefill 实例上是否已有相同模型的 Group 正在运行; 若存在且未超过最大批大小 MAX_GPFSIZE (默认 8), 则合并处理。否则追加至负载最小的 Prefill 实例队列中, 按批大小为 1 顺序执行, 从而避免在 Prefill 阶段触发频繁的模式频繁切换。
2. **Decoding 阶段加权轮询算法 (Algorithm 2)**: 作者明确陈述 调度器维护一个动态旋转的模型工作列表。每个模型批次 i 分配一个连续解码时间配额 q_i 。配额计算依据公式 (2) 与 (3), 由全局扩缩容开销 c 、各模型单步耗时 t_k 以及 TBT SLO 截止时间 d 共同决定, 确保在配额 q_i 内生成的 Token 能在客户端缓冲区支持下不发生 SLO 违约, 同时释放出的松弛时间用于掩盖下一个模型的上线开销。

前排论文大图 方法流水线 呼应: 第 2 节: 基本 LLM 推理与 SLO 定义

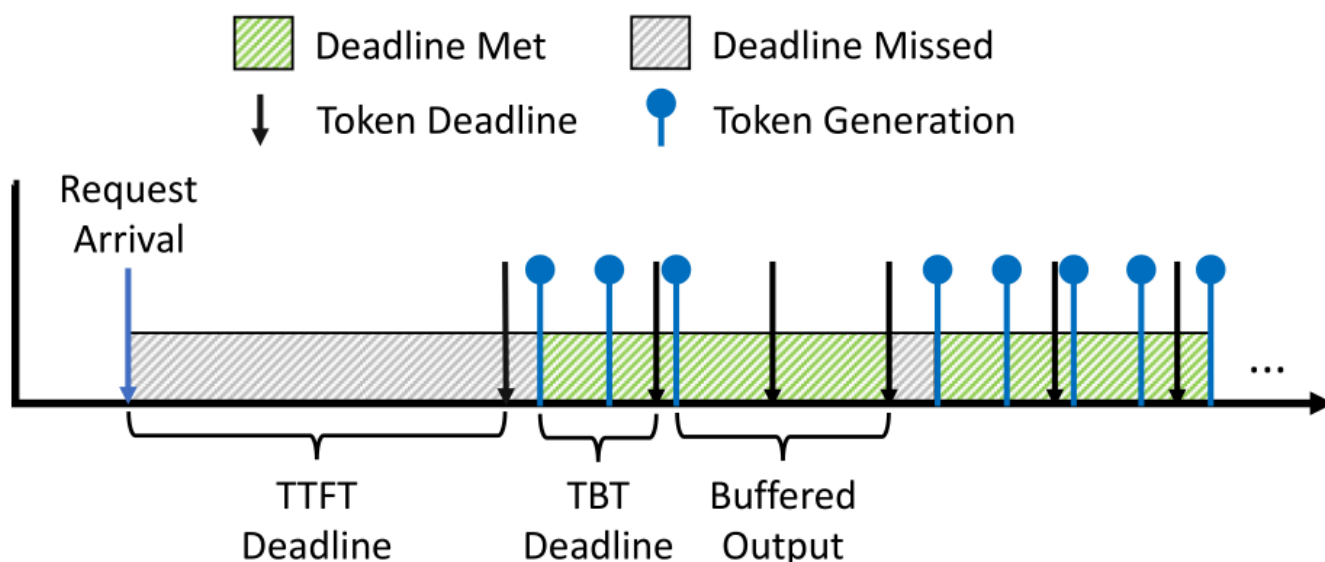


Figure 3. Overall token generation process of a request. SLO attainment is defined as the percentage of token generation times that meet their deadlines, i.e., the proportion of the green area.

图组解读

- **图组结论**: 解耦首 Token 延迟(TTFT)与 Token 间延迟(TBT), 按 Token 按时率计算 SLO 达成率。
- **论证关系**: 明确 Token 级评估机制, 支撑 Prefill/Decoding 解耦及利用松弛时间掩盖开销的设计。
- **适用范围**: 基于 Token 粒度的 SLO 评估模型。

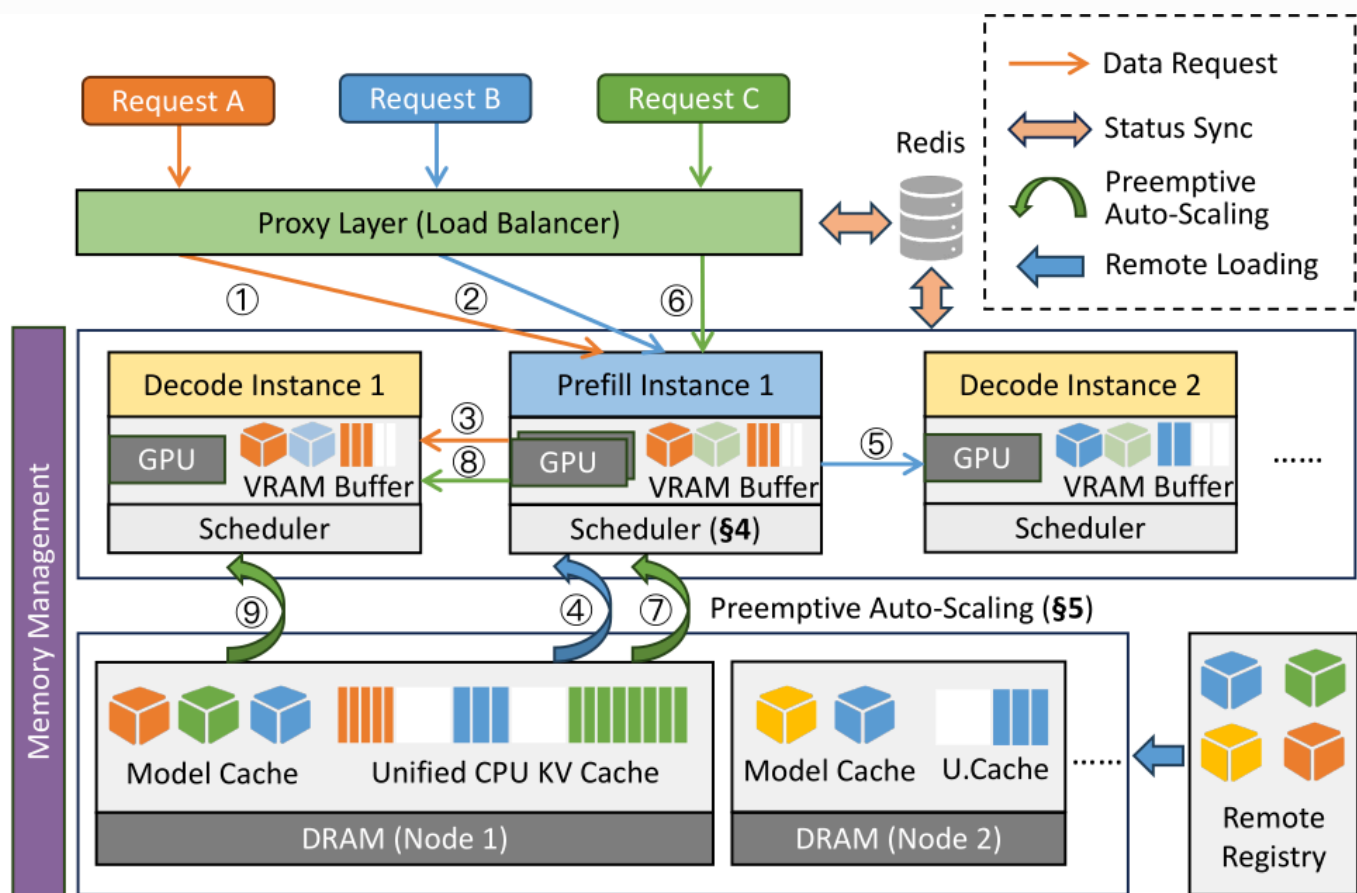


Figure 5. System overview of Aegaeon.

图组解读

- **图组结论：**架构清晰划分为 Proxy 控制层、解耦的 Prefill/Decoding 池及 DRAM/VRAM 显式内存管理层。
- **论证关系：**展示全栈架构，证明物理解耦与分级缓存能解决 Token 级调度与亚秒级扩缩容开销。
- **适用范围：**覆盖 Aegaeon 单节点与多节点部署架构。

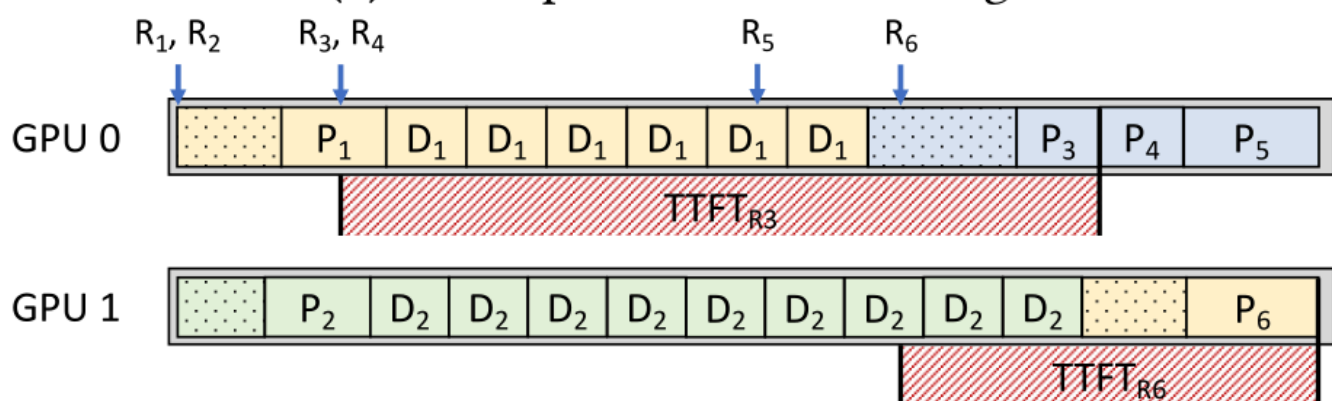
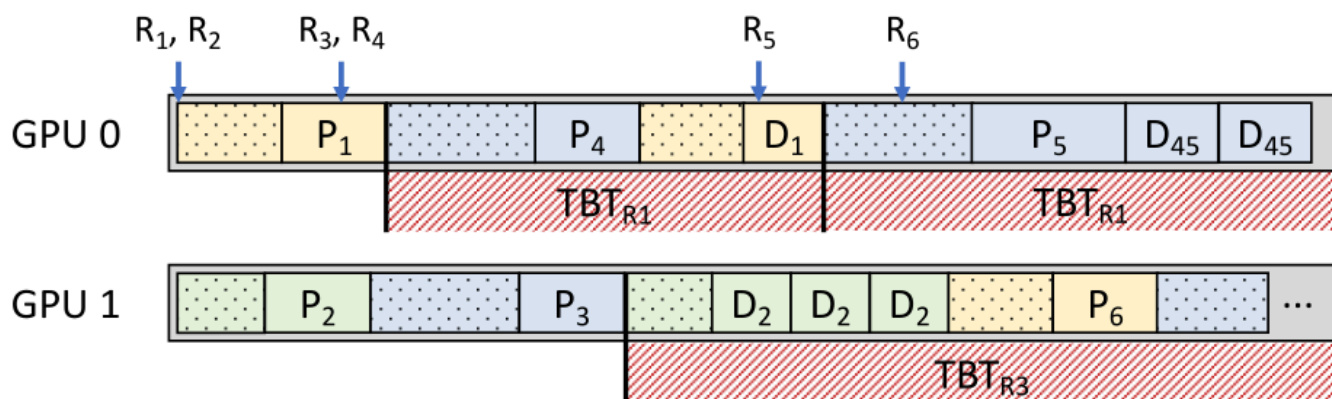
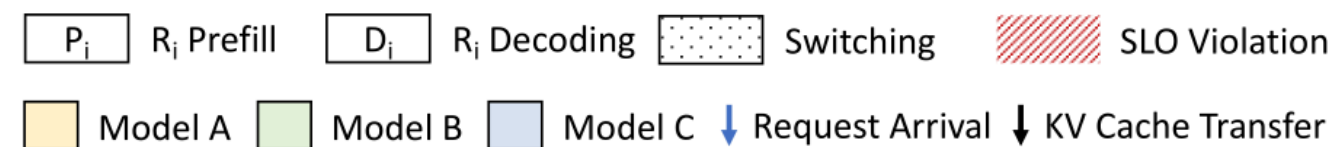


Figure 6. Exemplar token-level schedules on two GPUs. Prefill-first scheduling (a) and decoding-first scheduling (b) both lead to SLO violations. Disaggregated scheduling (c) separates prefill and decoding jobs to enable balanced token generation.

图组解读

- **图组结论：** Prefill 优先导致 TBT 违约，Decoding 优先导致 TTFT 违约，解耦调度实现零违约。
- **论证关系：** 证明将 GPU 池拆分为独立 Prefill 与 Decoding 实例的必要性，消除计算与访存任务干涉。
- **适用范围：** 2 卡 GPU 多模型 Token 级调度场景。

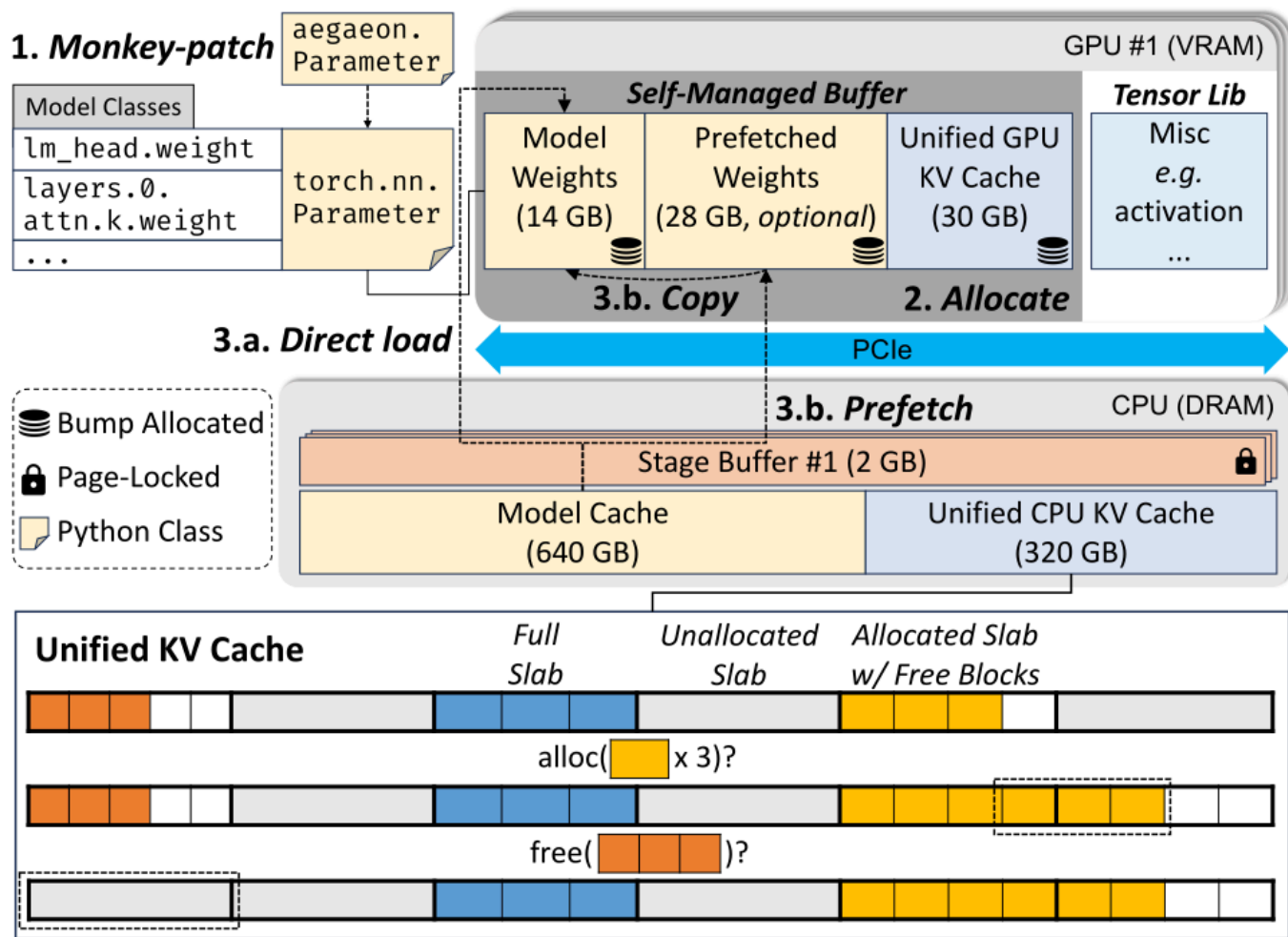


Figure 9. Explicitly managed memory in Aegaeon, with exemplar memory sizes in brackets. Relevant steps for scaling up a model and operations on the unified KV cache are also illustrated.

图组解读

- **图组结论：** 自研 VRAM 凸块分配器预留连续内存，配合 DRAM Model Cache 及统一 KV Cache Slab 切分。
- **论证关系：** 绕过 PyTorch 分配器消除 GC 停顿，并将多模型混用下的显存碎片率控制在 20% 以下。
- **适用范围：** 单 GPU 实例及 Host 内存环境下的内存管理。

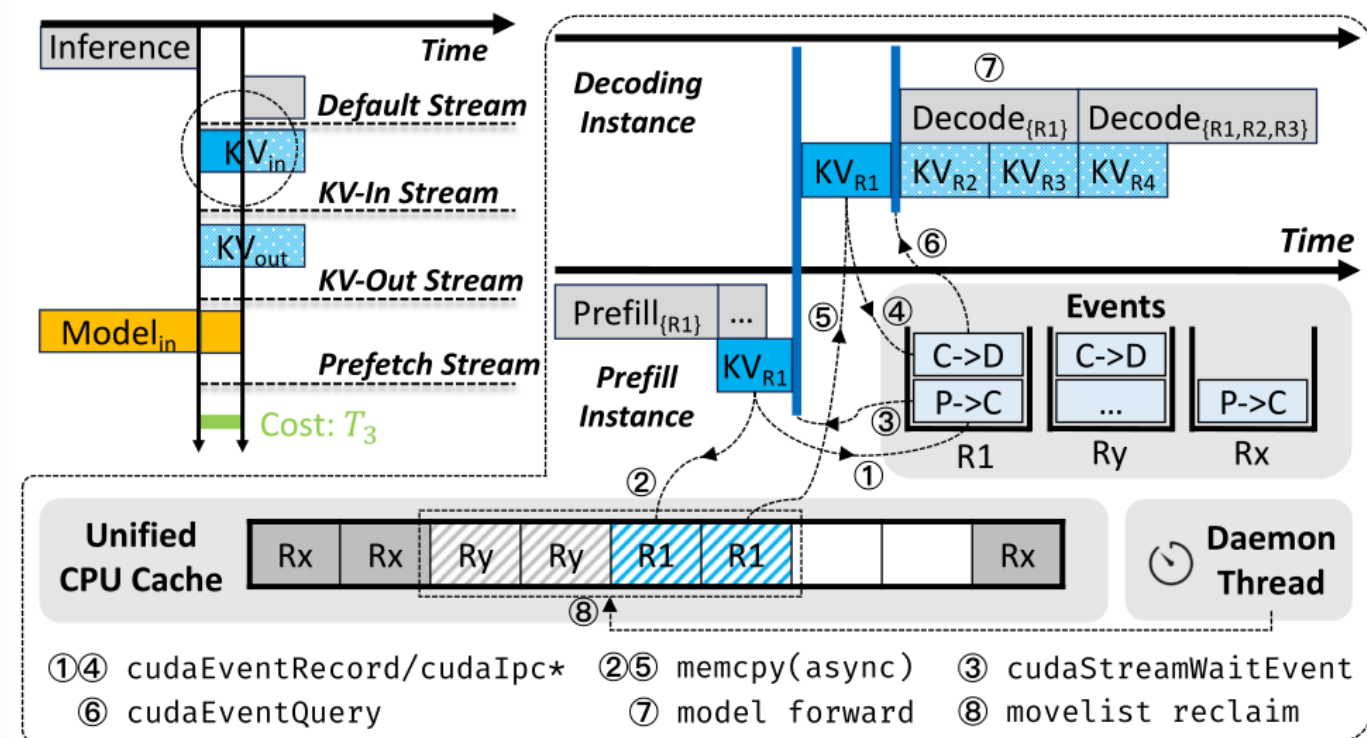


Figure 10. Efficient scaling in Aegaeon, with fine-grained KV cache synchronization. Dotted boxes indicate non-critical operations, and striped boxes indicate cache blocks in move lists.

图组解读

- **图组结论：**利用 4 个 CUDA Stream、CUDA Event 及 IPC 传递，实现 Prefill 到 Decoding 无锁同步。
- **论证关系：**解耦 CPU 控制流与 GPU 数据流，将扩缩容关键路径压缩至极短耗时，支撑亚秒级上线。
- **适用范围：**基于 CUDA 架构的多 Stream 与 IPC 同步。

7. 为什么它可能有效

基于文中逻辑的推断 下表连接 Aegaeon 的核心机制、预期技术收益以及生效所需的必要前提：

机制设计	预期技术收益	需满足的物理/逻辑条件
Prefill / Decoding 物理解耦	消除 Prefill 突发对 Decoding TBT 的干涉，让 TTFT 与 TBT 可以分别独立优化。	PCIe / 跨节点网络具备足够的带宽传输 KV Cache 状态。
Token 粒度加权轮询调度	消除队头阻塞，让挂起模型获得极低的 TTFT 排队延迟。	客户端具备缓冲区暂存 Token，且 TBT SLO 允许积累松弛时间 $n(d - t)$ 。
组件复用 (Component Reuse)	削减 80% 以上的推理引擎初始化延迟（从 26.9 秒降至 0.8 秒）。	模型架构在算子与 Executor 层面具有通用性（如均基于 Transformer 库）。
VRAM 凸块分配器 (Bump Allocator)	重置指针代替 empty_cache()，完全消除 Python 显存 GC 停顿。	模型权重与 KV Cache 的内存生命周期严格可预测。
统一 KV Cache Slab 分配器	将多模型混用下的 DRAM/VRAM 内存碎片率控制在 20% 以下。	KV Cache 块尺寸可按固定 Block/Slab 规格归一化切分。
CUDA Event 细粒度同步	跨进程/跨 GPU 传输 KV Cache 无锁流水线化，实现亚秒级甚至近零延迟上线。	硬件支持 CUDA Event API 与 CUDA IPC 句柄跨进程传递。

8. 实验与指标：证据是否支撑主张

8.1 实验问题与判定标准 **作者明确陈述** 实验部分设计了三类主要验证路线，旨在回答四个核心问题：

1. **池化效果问题（Goodput / SLO 达成率）：**在相同 GPU 数量下，Aegaeon 能否比现有系统支持更高倍数的 RPS 和更多数量的模型？[判定标准：SLO 达成率保持在 $\geq 90\%$ 时的最大模型数与最大 RPS]。

2. **开销削减问题 (Microbenchmarks)**：Aegaeon 的全栈优化是否真正将抢占式扩缩容开销降低至亚秒级？[判定标准：扩缩容延迟 CDF 分布与显存碎片率]。
3. **鲁棒性与扩展性问题**：在变体数据集、低端硬件及超大模型上，系统优势是否依旧存在？[判定标准：不同数据集与硬件上的 Goodput 曲线]。
4. **生产实用性问题**：真实生产集群中能否大幅节省 GPU 硬件开销？[判定标准：阿里云百炼部署的 GPU 数量缩减比例与利用率变化]。

8.2 数据、任务、对比与运行设置

- **测试数据集**：ShareGPT 真实对话数据集；变体数据集 ShareGPT-ix2 (输入长度翻倍)、ShareGPT-ox2 (输出长度翻倍)；生产环境真实请求日志。
- **硬件测试床**：
 - ▶ 实验室主集群：2 个节点共 16 张 NVIDIA H800 80GB GPU，PCIe 4.0 总线，2TB DDR5 Host 内存，192 CPU 核心。
 - ▶ 实验室补充集群：4 张 NVIDIA A10 24GB GPU (测试低显存扩展性)。
 - ▶ 生产集群：阿里云百炼跨区域集群，213 张 NVIDIA H20 GPU，托管 47 个模型 (参数量 1.8B 至 72B)。
- **对比基线系统**：
 - ▶ ServerlessLLM：SOTA 请求级自动扩缩容系统。
 - ▶ ServerlessLLM+：引入 Oracle SJF (最短作业优先) 调度的扩展变体。
 - ▶ MuxServe：SOTA 静态多路复用 GPU 池化系统。
- **服务 SLO 设置**：默认设置 TTFT SLO 为 10 秒，TBT SLO 为 100ms。

8.3 指标字典与对应公式 (1) 预期活跃模型数期望值 $\mathbb{E}[m]$

- **指标与方向**：预期活跃模型数期望值， $\mathbb{E}[m]$ ，测量系统在任意时刻至少有一个请求在运行的模型总数，单位：个，取值范围 $[0, M]$ ，越低越好。
- **定义与公式**：

$$\mathbb{E}[m] = M \cdot (1 - e^{-\lambda T})$$

- **来源层级**：原文公式 (编号：Eq. 1 / 第 4 页)。
- **实验用途**：用于理论证明“请求级自动扩缩容方法存在资源池化理论上限”。
- **读数与边界**：在 $M = 100, \lambda = 0.037, T = 16.79 \text{ s}$ 条件下，理论值与仿真值均落于 46.55 个模型，证明请求级扩缩容需预留大量 GPU。

(2) 解码阶段轮询时间配额 q_i

- **指标与方向**：解码阶段轮询时间配额， q_i ，测量调度器分配给第 i 个模型批次的连续解码时间，单位：秒，取值范围 $(0, Q_{\text{MAX}}]$ ，需按 SLO 动态调整。
- **定义与公式**：

$$q_i = \frac{c}{n_i \cdot \left(\alpha - \sum_k \frac{1}{n_k} \right)}$$

- **来源层级**：原文公式 (编号：Eq. 2 / 第 7 页)。
- **实验用途**：用于 Decoding 阶段 weighted round-robin 调度器确定各模型批次单次连续运行时间。
- **读数与边界**：根据实时扩缩容开销 c 与容忍步数 n_k 动态计算，上限 $Q_{\text{MAX}} = 4 \text{ 秒}$ 。

(3) SLO 达成率倒数估计值 α

- **指标与方向**：SLO 达成率倒数估计值， α ，评估当前模型集合在给定开销下的 SLO 达成倒数，无量纲，取值范围 $[0.5, +\infty)$ ，越小越好。
- **定义与公式**：

$$\alpha = \max \left(\frac{c}{\min_k(n_k) \cdot Q_{\text{MAX}}} + \sum_k \frac{1}{n_k}, 0.5 \right)$$

- **来源层级**：原文公式 (编号：Eq. 3 / 第 7 页)。
- **实验用途**：作为控制变量参与时间配额 q_i 的实时计算。
- **读数与边界**：当 $\alpha = 0.5$ 时表明系统拥有极高的延迟容忍裕度。

(4) 端到端 SLO 达成率 (SLO Attainment)

- **指标与方向**：端到端 SLO 达成率，SLO Attainment，测量满足 TTFT 与 TBT 截止时间的 Token 生成百分比，单位：%，取值范围 $[0\%, 100\%]$ ，越高越好 (合格线 $\geq 90\%$)。
- **定义与公式**：
 - ▶ 推理依据：原文事实：作者定义“SLO attainment as the percentage of token generation times that meet their deadlines” → 推导关系 → 指标含义。

$$\text{SLO Attainment} = \frac{\text{Count}(\text{TTFT} \leq \text{TTFT}_{\text{target}}) + \text{Count}(\text{TBT} \leq \text{TBT}_{\text{target}})}{\text{Total Generation Steps}} \times 100\%$$

- **来源层级**：推导关系 (非原文公式) **[基于文中逻辑的推断]**。
- **实验用途**：作为端到端性能与容量评估的核心主指标 (图 11、12、13)。

- **读数与边界**: 在 RPS=0.1 下, Aegaeon 在 70 个模型时依然维持 90% 以上的达成率。

(5) GPU 资源节省率 (GPU Resource Saving)

- **指标与方向**: GPU 资源节省率, 测量部署 Aegaeon 后减少的 GPU 数量比例, 单位: %, 取值范围 [0%, 100%], 越高越好。
- **定义与公式**:
 - 推理依据: 原文事实: 作者陈述 “reduces the number of GPUs required from 1,192 to 213, highlighting an 82% GPU resource saving” → 推导关系 → 指标含义。

$$\text{GPU Resource Saving} = \frac{N_{\text{baseline}} - N_{\text{Aegaeon}}}{N_{\text{baseline}}} \times 100\%$$

- **来源层级**: 推导关系 (非原文公式) **[基于文中逻辑的推断]**。
- **实验用途**: 评估生产环境部署收益 (§ 7.5)。
- **读数与边界**: 生产实际读数为 82.13% (节省 979 张 GPU)。

(6) 内存/显存碎片率 (Memory Fragmentation)

- **指标与方向**: 内存/显存碎片率, 测量系统未利用显存与峰值分配显存的比值, 单位: %, 取值范围 [0%, 100%], 越低越好。
- **定义与公式**:
 - 推理依据: 原文事实: 作者说明 “calculated as the ratio of unused memory to peak allocated memory” → 推导关系 → 指标含义。

$$\text{Fragmentation} = \frac{\text{Unused Memory}}{\text{Peak Allocated Memory}} \times 100\%$$

- **来源层级**: 推导关系 (非原文公式) **[基于文中逻辑的推断]**。
- **实验用途**: 评估统一 KV Cache Slab 分配器的显存空间利用效率 (图 16)。
- **读数与边界**: Aegaeon 的显存碎片率控制在 20% 以下, 显著优于 vLLM 默认分配器。

8.4 主结果、消融与证据边界 比较工作与受批判限制地图

比较工作 / 路线	本文指出的具体 Limit / 失效条件	本文对应的设计模块	检验批判的实验 / 指标 / 图表
MuxServe [20] (静态多路复用)	受限于 GPU 显存容量硬限制, 单卡只能放置 2 个 14B 模型, 16 GPU 场景下无法承载超过 32 个模型。	Token 粒度抢占式自动扩缩容, 模型权重从 Host DRAM 按需上线。	图 11 / 图 12 : MuxServe 在每 GPU 超过 2 个模型时拒绝配置, 折线终止于 32 个模型。
ServerlessLLM [21] (请求级自动扩缩容)	仅在请求粒度扩缩容, 引发严重队头阻塞 (HOL Blocking), 在密集到达或长生成场景下 SLO 大面积违约。	Token 粒度抢占式扩缩容 + Prefill/Decoding 解耦调度。	图 11 / 图 12 : Aegaeon 承载 RPS 为 ServerlessLLM 的 2.5 倍, Goodput 提升 1.5–9 倍。
ServerlessLLM+ (Oracle SJF 变体)	依靠 SJF 虽缓解排队, 但在密集到达时频繁触发扩缩容, 带来巨大的开销, 性能甚至劣于普通 ServerlessLLM。	Grouped FCFS (Prefill) + Weighted Round-Robin (Decoding) 抑制无意义扩缩容。	图 11(b) : 高 RPS (0.5) 下 ServerlessLLM+ 的 SLO 达成率崩塌最快。
通用 vLLM 引擎 (未优化后端)	重复初始化开销大 (26.9s), 显存碎片引发频繁 GC 停顿, KV Cache 同步阻塞。	组件复用 + Bump Allocator + Slab Allocator + CUDA Event 细粒度同步。	图 7 / 图 15 : 将扩缩容总开销削减 97% (降至 1 秒以内)。

主张地图与证据计划

主张类型	论文提出的核心主张与证据（实验/表图）	证据强度与边界
效果主张 (Effectiveness)	Aegaeon 实现单 GPU 承载多达 7 个模型，Goodput 提高 1.5–9 倍。（证据：图 11、图 12 端到端实验）	文本与图表证据充分。 16 GPU 承载 70 模型且满足 90% SLO。
效果主张 (Production)	阿里云百炼部署节省 82% GPU 资源（1,192 降至 213 张 H20），GPU 平均利用率提升至 48.1%。（证据：§ 7.5 生产部署数据）	生产实测证据充分。 包含 70 小时、47 个模型的真实运行监控。
机制主张 (Mechanism)	扩缩容全栈优化消除 97% 开销；预取机制使 50% 的扩缩容达到接近零延迟。（证据：图 7、图 15 开销消融与 CDF）	微基准证据充分。 CDF 图直观展示亚秒级开销分布。
必要性主张 (Necessity)	请求级自动扩缩容受限于定理 3.1 的活跃模型期望数 $\mathbb{E}[m]$ ，必须使用 Token 粒度扩缩容。（证据：图 4 仿真与定理 3.1）	理论+仿真证据充分。 公式推导与实验对比互相印证。
鲁棒性主张 (Robustness)	在 A10 24GB 低端 GPU 以及 72B (TP=4) 超大模型上均表现出强拓展性。（证据：§ 7.4 图 17 扩展性实验）	文本证据充分。 覆盖不同硬件配置与并行度。
适用范围 (Scope/Limit)	在极严苛延迟场景 (TTFT 2s / TBT 20ms) 下，Aegaeon 收益下降，静态多路复用重获优势。（证据：图 13(c) SLO 敏感性实验）	作者明确陈述并由实验证明。 清晰给出了方法生效的下限边界。

- **实验设置关键补充：** 实验室实验使用 16 张 H800 80GB GPU，按 6 张 Prefill + 10 张 Decoding 划分为物理解耦池。
- **指标与判定补充：** SLO Attainment 门槛固定为 90%，在 RPS=0.1 时 Aegaeon 解码池（10 GPUs）成功承载 70 个模型，达到 7 模型/GPU 的密度。
- **证据缺口说明：** 跨节点管线并行（PP）场景下的细粒度 KV Cache 同步表现文中未说明。

9. 图表解读与论证关系

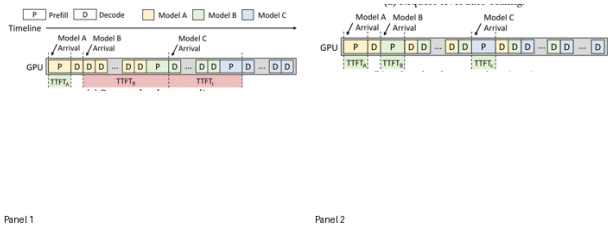


Figure 2. Different auto-scaling granularities for sharing one GPU instance between three models.

图组解读 对比基线 呼应：第 4 节：自动扩缩容路线局限与队头阻塞

- **图组结论：** 请求级扩缩容引发严重队头阻塞与 TTFT 违约；Token 级扩缩容可随时插队，保持极短 TTFT。
- **论证关系：** 直观验证定理 3.1，阐明请求级与 Token 级扩缩容机制差异，支撑引入 Token 级抢占的核心洞见。
- **适用范围：** 3 模型共享单张 GPU 实例的抢占调度概念模型。

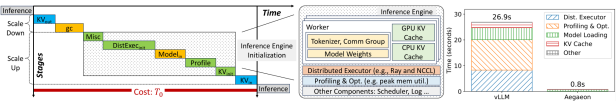


Figure 7. Left: default preemptive auto-scaling process for an inference engine. Middle and Right: typical composition of an LLM inference engine, and the latency breakdown of its initialization (both before and after our optimization).

图组解读 消融验证 呼应：第 6 节：高效抢占式自动扩缩容引擎优化

- **图组结论：** vLLM 默认初始化耗时 26.9 秒(DistExec 与 Profile 占超 80%); Aegaeon 通过组件复用削减至 0.8 秒。
- **论证关系：** 消融验证组件复用消除 80%+启动开销，将扩缩容延迟削减 97%，实现亚秒级抢占。
- **适用范围：** 13B 模型(TP=2)在 PCIe 4.0 下的初始化测试。

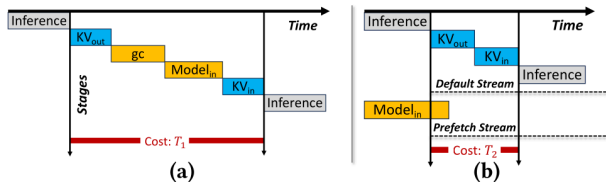


Figure 8. Preemptive scaling with optimizations in § 5.1 and § 5.2. (a) w/ component reuse; (b) also w/ explicit memory management.

图组解读 消融验证 呼应：第 6 节：显式内存管理与模型预取消融

- **图组结论**：叠加凸块分配器与 CUDA 流异步预取后，扩缩容关键路径从 T1 缩短至仅剩 KV Cache 换入换出的 T2。
- **论证关系**：消融验证凸块分配器消除 GC 停顿与预取流隐藏模型加载开销对缩短关键路径的独立贡献。
- **适用范围**：具备足够显存容纳预取模型的 GPU 环境。

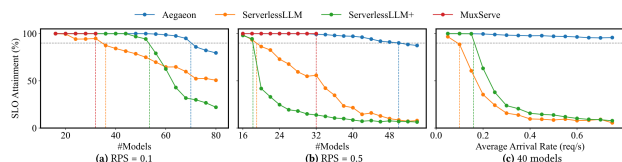


Figure 11. End-to-end SLO attainment under varying RPS with the ShareGPT dataset.

图组解读 实验结论 呼应：第 8 节：端到端性能与 SLO 达成率主张

- **图组结论**：RPS=0.1 下单 GPU 承载 7 模型；高 RPS(0.5)下容量为 ServerlessLLM 的 2.5 倍。
- **论证关系**：为效果主张提供端到端实验证据，证明系统超越静态多路复用与请求级扩缩容。
- **适用范围**：16 张 H800 GPU(6 Prefill + 10 Decoding)环境。

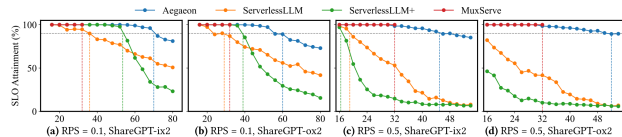


Figure 12. End-to-end SLO attainment under varying RPS with alternative datasets. x-axis: number of models.

图组解读 实验结论 呼应：第 8 节：不同负载下的鲁棒性主张

- **图组结论**：在长输入(ix2)与长输出(ox2)数据集上，Aegaeon 的 90% SLO 模型上限仍达 50-70 个，基线崩塌。
- **论证关系**：验证系统在不同负载下的强鲁棒性，以及 Grouped FCFS 与加权轮询算法的正确性。
- **适用范围**：ShareGPT-ix2 与 ShareGPT-ox2 变体数据集。

表格证据：Table 1. 消融验证 呼应：第 6 节：统一 KV Cache Slab 分配器

- **图组结论**：展示不同模型单 Token 的 KV Cache 维度与尺寸差异。
- **论证关系**：说明多模型 KV Cache 异构引发内存碎片，支撑统一 Slab 分配器设计。
- **适用范围**：覆盖 vLLM 常见开源 LLM 架构的 KV Cache 规格。

Model	KV Cache Shape	KV Cache Size
Qwen-7B [38]	(32, 2, 32, 128)	512 KB
InternLM2.5-7B-chat [26]	(32, 2, 8, 128)	128 KB
LLaMA-13B [42]	(40, 2, 40, 128)	800 KB
Qwen-72B [38]	(80, 2, 64, 128)	2560 KB

表格证据：Table 2. CUDA event APIs used in Aegaeon. 消融验证 呼应：第 6 节：细粒度 KV Cache 同步机制

- **图组结论**：汇总 Aegaeon 使用的底层 CUDA Event API 及 IPC 句柄跨进程传递方法。
- **论证关系**：提供 CUDA Event 无锁异步同步调度的具体 API 接口支撑。
- **适用范围**：基于 CUDA 驱动层与 IPC 跨进程通信 API。

API	Description
cudaEventRecord(event, stream)	Capture the current work in stream into event .
cudaEventQuery(event)	Query the completion status of the work captured in event.
cudaStreamWaitEvent(stream, event)	Make all future work submitted to stream wait for the work captured in event.
cudaIpcGetEventHandle(handle, event)	Get a handle for event for interprocess access.
cudaIpcOpenEventHandle(event, handle)	Reconstruct event from an interprocess handle.

表格证据: Table 1. Symbols used in the latency modeling. 消融验证 呼应: 第 3 节: 延迟建模与数学定义

- **图组结论:** 汇总延迟建模与加权轮询调度算法中使用的数学符号定义。
- **论证关系:** 为定理 3.1 证明与 Decoding 阶段时间配额计算公式(2)(3)提供准确的符号含义对照。
- **适用范围:** 覆盖论文理论推导与调度算法中的全部建模符号。

Symbol	Meaning
h	hidden size of the LLM
m	FFN intermediate size
l_k	input length of the k -th request in the batch
t	number of tokens in the batch ($t = \sum_{i=0}^{B-1} l_i$)
t_2	squared sum of the input lengths ($t_2 = \sum_{i=0}^{B-1} l_i^2$)
b	block size in the FlashAttention [18] kernel

10. 完整因果推理树

- 根问题: 并发 LLM 服务中专属 GPU 分配在冷模型上导致严重资源浪费, 在热模型上无法应对突发流量。
 - 旧方法限制:
 - 静态多路复用受限于物理显存容量, 单 GPU 最多承载 2-3 个模型。
 - 请求级自动扩缩容受限于定理 3.1 导出的高活跃模型数 $\mathbb{E}[m]$, 引发严重的队头阻塞。
 - 核心洞见/假设:
 - 洞见 1: 按 Token 粒度执行抢占式自动扩缩容可消除队头阻塞。
 - 洞见 2: Prefill/Decoding 物理解耦可消除计算密集与访存密集任务的相互干涉。
 - 假设 1: Decoding 阶段积累的松弛时间可用于掩盖扩缩容开销。
 - 假设 2: 引擎组件复用与显式内存管理可将扩缩容开销降至亚秒级。
 - 方法模块:
 - 模块 1: Prefill / Decoding 物理解耦 Serving 实例池。
 - 模块 2: Prefill 阶段 Grouped FCFS 调度算法 (Algorithm 1)。
 - 模块 3: Decoding 阶段加权轮询调度算法 (Algorithm 2)。
 - 模块 4: 高效抢占式扩缩容引擎 (组件复用 + Bump Allocator + Model Cache/Prefetch + Slab Allocator + CUDA Event 细粒度同步)。
 - 可验证预测:
 - 预测 1: 扩缩容开销大幅削减 90% 以上, 达到亚秒级。
 - 预测 2: 单 GPU 承载模型数量大幅提升至 5-7 个, Goodput 提升 1.5-9 倍。
 - 预测 3: 生产集群可用更少 GPU 服务相同数量的模型。
 - 指标与实验:
 - 评估指标: 扩缩容延迟 CDF、端到端 SLO Attainment、Goodput、GPU 资源节省率、显存碎片率。
 - 实验设置: 16 张 H800 GPU 测试床 (ShareGPT 数据集) 与 213 张 H20 生产集群。
 - 图表证据:
 - 图 7 / 9 / 15: 证实扩缩容延迟从 26.9s 降至 1s 以内 (削减 97%)。
 - 图 11 / 12: 证实单 Decoding GPU 承载 7 个模型, Goodput 提升 1.5-9 倍。
 - § 7.5 部署数据: 证实生产环境显卡节省 82% (1,192 降至 213 张)。
 - 结论与适用边界:
 - 结论: Token 粒度抢占式扩缩容与解耦调度成功破解了并发多模型 Serving 的池化难题。
 - 适用边界: 在极端低延迟 (TTFT 2s / TBT 20ms) 场景下收益下降; 小显存 GPU (如 A10 24GB) 上需禁用模型预取。

11. 结论、贡献与边界

11.1 核心贡献与边界划分

1. **理论与机制创新**: **作者明确陈述** 首次量化了并发 LLM 服务中请求级自动扩缩容的理论限制 (定理 3.1), 并提出了 Token 粒度自动扩缩容与 Prefill/Decoding 物理解耦调度的系统架构。
2. **工程与全栈优化**: **作者明确陈述** 针对 Token 级调度的开销挑战, 设计了包含组件复用、VRAM 凸块分配器、模型预取、统一 Slab 管理以及 CUDA Event 异步同步的全栈优化方案, 将扩缩容延迟削减 97%。
3. **真实落地验证**: **实验支持** 在实验室测试床中实现了单 GPU 承载 7 个模型与 1.5–9 倍 Goodput 提升; 在阿里云百炼生产部署中实现了 82% 的 GPU 硬件节省。

11.2 审稿式风险检查

- **贡献新意 (Novelty)**: 强。将 Serverless 自动扩缩容推进到 Token 细粒度, 且提出了针对 LLM 引擎全栈开销的完整系统化解答 **基于文中逻辑的推断**。
- **写作/可复现性 (Reproducibility)**: 中等偏强。论文公式推导严谨, 算法细节 (Algorithm 1 & 2) 与内存结构图解详尽; 但底层大量使用了 PyTorch 内部 Monkey-patching 与 CUDA IPC 私有接口, 文中未说明代码开源仓库, 复现门槛较高 **基于文中逻辑的推断**。
- **效果大小 (Effect Size)**: 极强。实验室 1.5–9 倍 Goodput 提升与生产集群 82% 的物理显卡节约均属于重大突破 **实验支持**。
- **实验覆盖 (Experimental Coverage)**: 强。覆盖了微基准、变体数据集、扩展硬件 (A10)、72B 大模型以及 70 小时生产实测 **实验支持**。
- **方法设计与假设风险**: **作者明确陈述** 在 SLO 极度严苛 (TTFT 2s / TBT 20ms) 的场景下, 松弛时间不足以掩盖开销, 导致系统优势丧失。当前证据未显示其他重大设计缺陷。

12. 术语与第一性原理学习路径

12.1 核心术语 (1) Token 粒度自动扩缩容 (Token-Level Auto-Scaling)

- **最小定义**: 在 LLM 生成单个字符/词单元 (Token) 的微观时间步上执行模型的下线与上线抢占操作。
- **第一性原理角色**: 打破模型服务时间 T 在宏观时间轴上的连续物理约束, 将排队时间从 $O(T)$ 压缩至 $O(t_{\text{step}})$ 。
- **本文位置**: 本文最核心的创新点与系统设计基石。

(2) Prefill 与 Decoding 解耦 (Prefill/Decoding Disaggregation)

- **最小定义**: 将 GPU 实例池物理拆分为专门处理 Prompt 的 Prefill 池与专门处理字符生成的 Decoding 池。
- **第一性原理角色**: 隔离计算密集型任务 (矩阵乘法计算界) 与访存密集型任务 (显存带宽访存界) 对 GPU 执行单元的争抢。
- **本文位置**: 本文调度架构 (§ 4) 的核心设计选择。

(3) 凸块分配器 (Bump Allocator)

- **最小定义**: 通过单向移动线性指针来连续分配显存, 并通过一次性重置指针来全量释放显存的分配器。
- **第一性原理角色**: 利用内存分配的单调性约束, 消除复杂链表查找与显存碎片垃圾回收 (GC) 停顿。
- **本文位置**: 用于接管 PyTorch 默认显存分配, 消除扩缩容过程中的 GC 延迟。

(4) 统一 KV Cache Slab 分配器 (Unified KV Cache Slab Allocator)

- **最小定义**: 将 DRAM/VRAM 划分为固定尺寸的 Slab 块, 按 Block 粒度统一管理不同模型 Attention KV Cache 的内存分配器。
- **第一性原理角色**: 解决多模型混合运行下由于 KV Cache Shape (头数、维度) 差异造成的内存空间碎片化。
- **本文位置**: 用于将多模型并发下的显存碎片率降至 20% 以下。

(5) CUDA Event 异步同步 (CUDA Event Asynchronous Synchronization)

- **最小定义**: 利用 NVIDIA CUDA 驱动层的 Event API 与 IPC 句柄, 在不同进程或 Stream 之间实现无锁通知与依赖等待。
- **第一性原理角色**: 解耦 CPU 控制流与 GPU 数据流, 实现异步流水线完全重叠。
- **本文位置**: 用于 Prefill 到 Decoding 阶段 KV Cache 的细粒度、亚秒级移交。

12.2 第一性原理学习路径 理解本文前建议补充的 4 个基础概念, 按物理与逻辑依赖顺序排列:

1. **基本对象 (显存与 KV Cache)**: 理解 LLM 在生成字符时为何需要维持 KV Cache (键值缓存, 避免重复计算历史 Token), 以及显存容量与带宽如何成为 Serving 的硬物理瓶颈 **背景知识**。
2. **系统物理约束 (TTFT 与 TBT SLO)**: 理解用户体感延迟被拆解为首字延迟 (TTFT) 与打字间隔 (TBT), 以及客户端缓冲区如何提供延迟容忍的“松弛时间” **背景知识**。
3. **运行机制 (Prefill 与 Decoding 的计算特征差异)**: 理解为什么 Prefill 是计算密集型 (大 GEMM), 而 Decoding 是访存密集型 (单 Token GEMV), 从而直观理解两者混合运行时的干涉冲突 **背景知识**。
4. **指标与系统行为 (排队论与 HOL Blocking)**: 理解排队论中的队头阻塞现象, 以及为何将调度粒度从“请求级”切碎到“Token 级”能够在物理上消除短任务等待长任务的排队延迟 **基于文中逻辑的推断**。